# Android Project Structure

An Android application consists of multiple components that work together to create a functional app. The structure follows a well-defined architecture, including UI, business logic, and data storage.

**Key Components of an Android App:**

1. **Manifest File (`AndroidManifest.xml`)**

2. **Java/Kotlin Code (`src/main/java/`)**

3. **Resources (`res/`)**

4. **Gradle Build Files (`build.gradle`)**

5. **Application Components** (Activities, Services, Broadcast Receivers, Content Providers)

ShrineCompose main Pixel app

Android

- app
  - manifests
  - kotlin
    - com.example.shrinecompose
      - ui.theme
      - Backdrop.kt
      - CartBottomSheet.kt
      - Catalog.kt
      - Data.kt
      - FirstAddCartItem.kt
      - MainActivity.kt
      - ShrineApp.kt
      - Utils.kt
    - com.example.shrinecompose (andr
    - com.example.shrinecompose (test)
  - java (generated)
  - res
  - res (generated)
- Gradle Scripts
  - build.gradle (Project: ShrineCompose)
  - build.gradle (Module :app)
  - proguard-rules.pro (ProGuard Rules fo
  - gradle.properties (Project Properties)
  - gradle-wrapper.properties (Gradle Ver
  - local.properties (SDK Location)
  - settings.gradle (Project Settings)

MainActivity.kt | ShrineApp.kt | Utils.kt | CartBottomSheet.kt

```kotlin
/ Copyright 2022 Google LLC .../

package com.example.shrinecompose

import ...

Chris Sinco +1
@ExperimentalAnimationApi
@ExperimentalMaterialApi
class MainActivity : ComponentActivity() {
    Chris Sinco
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ShrineComposeTheme {
                ShrineApp()
            }
        }
    }
}

Chris Sinco
@ExperimentalAnimationApi
@ExperimentalMaterialApi
@Preview(showBackground = true)
@Composable
fun AppPreview() {
    ShrineComposeTheme {
```

app > src > main > kotlin > com > example > shrinecompose > MainActivity.kt > MainActivity 28:26 LF UTF-8 4 spaces

## I. Manifest File (`AndroidManifest.xml`)

- Located at the root of the project.
- Declares the app’s package name, components, permissions, and metadata.

# Example

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">
    <application
        android:allowBackup="true"
        android:label="MyApp">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

## Ii. Java/Kotlin Source Code (`src/main/java/`)

- Contains all the logic of the application.
- Organized into **Activities, ViewModels, Adapters, and Data Repositories**.

# Example

kotlin

```kotlin
package com.example.myapp
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

# Iii. Resources (`res/`)

Stores non-code assets such as UI layouts, images, and string values.

**Common Folders in `res/` Directory:**

1. `layout/` → XML files defining the UI structure.
2. `drawable/` → Images, icons, and vector graphics.
3. `values/` → Stores XML files for colors, strings, and styles.
4. `mipmap/` → Launcher icons in different resolutions.

## `Layout/` → XML files defining the UI structure.

xml

Copy Edit

```xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, Android!" />
</LinearLayout>
```

**`values/`** → Stores XML files for colors, strings, and styles.

- `strings.xml` (Stores text values)

xml Copy Edit

```xml
<resources>
    <string name="app_name">MyApp</string>
</resources>
```

- `colors.xml` (Defines color resources)

xml Copy Edit

```xml
<resources>
    <color name="primaryColor">#6200EE</color>
</resources>
```

# Iv. Gradle Build System

- Manages dependencies, configurations, and compilation settings.

1. Project-Level `build.gradle`

```
dependencies {
    classpath 'com.android.tools.build:gradle:7.0.0'
}
```

2. Module-Level `build.gradle` (app)

```
dependencies {
    implementation 'androidx.appcompat:appcompat:1.3.0'
    implementation 'com.google.android.material:material:1.3.0'
}
```

# UI Design Patterns

## 1 XML-Based UI

- Uses XML files to define layouts.

## 2 Jetpack Compose

- Uses Kotlin-based declarative UI framework.

# Android Project Structure

1. Manifest File (AndroidManifest.xml)
2. Java/Kotlin Files
3. XML Layouts
4. Resources (Drawable, Values, Mipmap)
5. Gradle Build Files

# Intents and Navigation

1. Explicit Intent (Navigate to specific activity)
2. Implicit Intent (Open external apps, URLs)